

El tablero donde las flechas deciden tu destino

^aJuan Alejo, Acosta; ^bLucas Ivan, Benchat Parra; ^cAmbar Noara, Martínez; ^dLautaro Emanuel, Sandoval; ^eRaúl Vicente Agustín, Verón; ^fLucas Maximiliano, Zurano Lang

^a Universidad Tecnológica Nacional F.R.Re, ISI K2.1, acostajuanalejo@gmail.com

^b Universidad Tecnológica Nacional F.R.Re, ISI K2.2, libenchat@hotmail.com

^c Universidad Tecnológica Nacional F.R.Re, ISI K2.2, ambarnoaramartinez64@gmail.com

^d Universidad Tecnológica Nacional F.R.Re, ISI K2.2, lautisando@gmail.com

^e Universidad Tecnológica Nacional F.R.Re, ISI K2.2, agustinveron883@gmail.com

^f Universidad Tecnológica Nacional F.R.Re, ISI K2.2, lucaslang2010@gmail.com

Resumen

En este artículo se presenta el trabajo en equipo realizado para el desarrollo de un juego de tablero en el que los jugadores avanzan siguiendo flechas cuya orientación cambia cada vez que una casilla es abandonada. Durante el análisis del diseño original se observó que la dinámica quedaba determinada desde el inicio de la partida, lo que reducía la sorpresa y hacía que el interés del jugador disminuyera con rapidez.

Con el objetivo de ofrecer una experiencia más atractiva, se creó una versión mejorada en la que se incorporan elementos de azar y decisiones que influyen en el transcurso del juego. Entre las modificaciones implementadas se incluyen minijuegos para definir quién inicia la partida y quién determina el tamaño del tablero, un sistema de turnos manuales, un límite de pasos y una interfaz visual clara y accesible. También se añadieron vidas extras ubicadas en casillas aleatorias, generando situaciones inesperadas y aumentando la competitividad entre los jugadores.

En conjunto, estas mejoras transforman el desarrollo previsible del juego original en una experiencia más dinámica y variable, capaz de mantener el interés hasta el final de cada partida.

1. Introducción

El trabajo presentado es expuesto como una versión ampliada y mejorada del Sistema Dinámico Repulsor, un juego de tablero. En el diseño original, el recorrido del tablero es efectuado a través de casillas que contienen flechas orientadas en alguna de las cuatro direcciones cardinales. Cada movimiento es condicionado por la dirección indicada y, al abandonarse una casilla, su flecha es rotada 90° en sentido horario, generándose así un sistema dinámico modificado con cada paso del jugador.

La implementación del proyecto es realizada utilizando Pharo, un entorno completamente orientado a objetos basado en Smalltalk. Por medio de este lenguaje, el comportamiento del tablero, de las flechas y de los jugadores es modelado aplicando conceptos fundamentales del paradigma, tales como abstracción, encapsulamiento, responsabilidad por clase, mensajería y control del estado mutable. Las clases diseñadas para representar los elementos del juego son organizadas de manera tal que la estabilidad del sistema sea mantenida al procesarse los movimientos, actualizarse el tablero y evaluarse las condiciones de finalización de la partida. “En

Smalltalk todo es un objeto, incluyendo clases base y tipo base. Esto significa que la potencia de Smalltalk, como un entorno de programación completo, se fundamenta en el envío de mensajes a objetos” (Joyanes Aguilar, 1996, p. 116).

La consigna es ampliada mediante la incorporación de nuevas mecánicas y elementos aleatorios cuyo objetivo es enriquecer la jugabilidad y evitar que las partidas resulten previsibles. Son añadidas decisiones interactivas para el jugador, variaciones en el comportamiento de las flechas, eventos de azar y ajustes dinámicos del entorno que influyen en la estrategia. Para integrar estas características sin comprometer la integridad del sistema, un modelo claro, escalable y coherente con el paradigma orientado a objetos es planificado.

El proceso de trabajo es llevado adelante mediante reuniones de equipo, iteraciones constantes sobre el diseño general, construcción de clases específicas y verificación continua del funcionamiento. De este modo, la teoría vista en la materia Paradigmas de Programación es articulada con la práctica del diseño orientado a objetos, permitiendo que el proyecto sea concluido como un juego funcional, estable y ampliado respecto de la versión inicial.

2. Desarrollo

2.1 Organización del Equipo y Planificación del Trabajo

El desarrollo del proyecto se inició con la organización del equipo de trabajo mediante la creación de un canal de comunicación grupal, ver Fig. 1, que permitió coordinar horarios, compartir archivos y establecer acuerdos generales. Para garantizar una planificación clara, se utilizó una pizarra digital en Trello, ver Fig. 2, en la cual se registraron las tareas principales, se distribuyeron responsabilidades y se mantuvo el control del avance. Esta herramienta facilitó una administración ordenada del proyecto y posibilitó realizar ajustes cuando surgieron nuevas necesidades durante la programación.

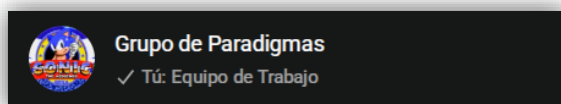


Fig. 1: Grupo de comunicación del equipo.

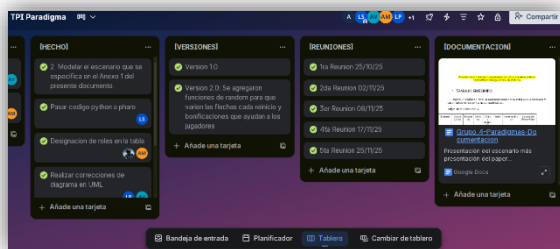


Fig. 2: Herramienta de Trello.

2.2 Prototipo Inicial en Python

Una vez establecida la organización interna, se dio comienzo a la construcción de un primer prototipo del sistema mediante el lenguaje Python, llevado a cabo en diferentes reuniones en el foro de discord, ver Fig. 3. Este prototipo tuvo como finalidad comprobar la lógica central del funcionamiento mediante la incorporación de elementos gráficos. Para ello fueron programadas las clases Tablero, Celda y Jugador, entre otras, lo que permitió evaluar el comportamiento de las direcciones, la rotación de las celdas y el desplazamiento dentro del tablero. La elección de Python para esta etapa se debió a su simpleza sintáctica, lo cual permitió detectar inconsistencias, corregir comportamientos no deseados y realizar ensayos rápidos antes de avanzar hacia el entorno definitivo.

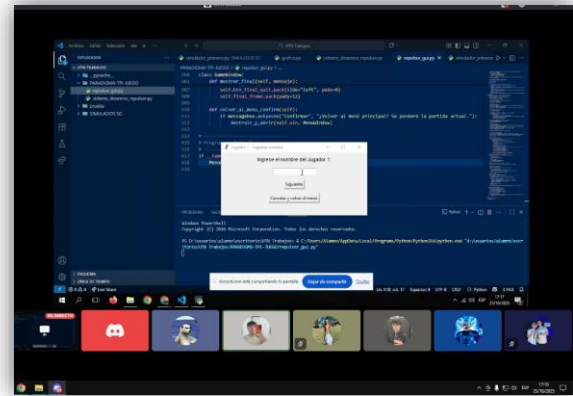


Fig. 3: Herramienta de comunicación Discord

2.3 Modelado de Clases y Diagrama UML

Durante la implementación de Pharo se elaboró el modelado del sistema mediante la definición completa de sus clases principales y relaciones. Este modelado se efectuó siguiendo los principios del paradigma orientado a objetos, lo que permitió asignar responsabilidades bien delimitadas a cada componente. En este diagrama se establecieron entidades tales como Tablero, Celda, Jugador, juegoEnEjecución, Minijuegos, ver Fig 4. "Al ser un lenguaje de modelado orientado a objetos, todos los elementos y diagramas en UML se basan en el paradigma orientado a objetos". (Joyanes Aguilar, 2008, p. 568).

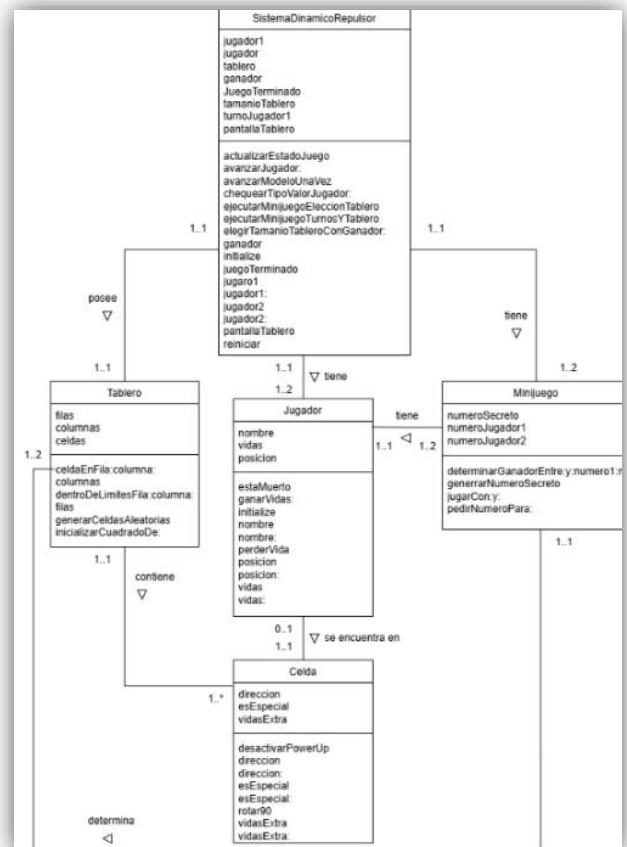


Fig. 4: Diagrama de clases UML

2.4 Implementación en Pharo

Una vez completado el modelado, el proyecto fue trasladado al entorno Pharo para desarrollar la versión final. La elección de Pharo respondió a su enfoque completamente orientado a objetos y a las facilidades que ofrece para inspeccionar estados internos, depurar dinámicas en tiempo real y visualizar la interacción entre objetos de manera directa. En este entorno se construyó una jerarquía de pantallas basada en una clase abstracta denominada Pantalla, la cual unificó el estilo visual, los métodos de navegación y la estructura común de la interfaz. Sobre esta base se definieron las pantallas PantallaInicial, ExplicacionJuego, PantallaInfoJugador, PantallaEleccionPosicion, PantallaTablero, y PantallaFinal. “La abstracción es la clave para diseñar un buen software” (Joyanes Aguilar, 1996, p. 11).

El Tablero fue programado como estructura central encargada de contener la matriz de celdas y administrar sus comportamientos internos, mientras que cada Celda fue diseñada para gestionar su dirección, color, estado especial, vidas y la presencia del jugador. Jugador almacena información esencial como nombre y cantidad de vidas. Finalmente, la clase JuegoEnEjecucion coordinó el funcionamiento general mediante la administración de turnos, la verificación de condiciones y la sincronización de todos los componentes.

A continuación, se realiza un detalle de estas clases y sus métodos, indicando sus funcionalidades y cómo contribuyen al funcionamiento del juego, ver Fig. 5.

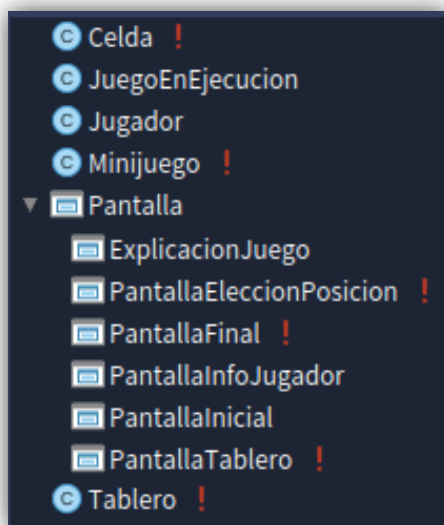


Fig. 5: Vista de todas las clases en pharo.

Clase Pantalla

La clase Pantalla fue diseñada como el punto de partida para definir las distintas pantallas/menús del juego. A pesar de que solo provee un único método de clase, «abrir», el cual ofrece una manera genérica de abrir las distintas pantallas con ciertos parámetros predefinidos, también es una descendiente de la clase SpPresenter, por lo que actúa además como la manera de crear interfaces a través del framework Spec, ver Fig. 6.

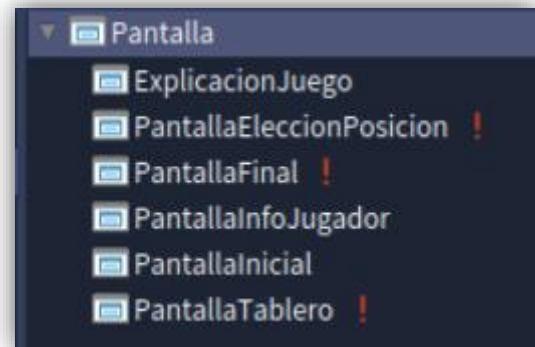


Fig. 6: Vista de clase pantalla.

Clase Tablero

La clase Tablero fue diseñada con la idea de contener la representación lógica del tablero, mientras que la representación visual fue delegada a otra clase. Los métodos «filas» y «columnas» permiten obtener el tamaño elegido por los jugadores por el tablero, «dentroDeLimitesFila:columna:» permite comprobar si un jugador se encuentra dentro de los límites del tablero, «celdaEnFila:columna:» permite obtener la celda ubicada en una posición específica, y «generarCeldasAleatorias» permite generar un tablero con celdas de distintas características, de acuerdo a las dimensiones elegidas por el ganador del minijuego utilizado para decidir cuáles serán dichas dimensiones, ver Fig. 7.

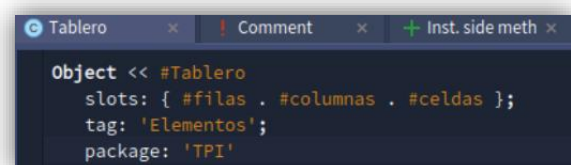
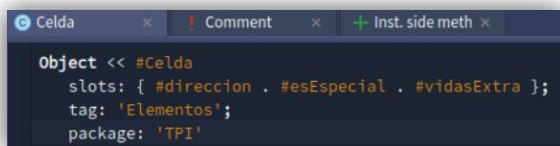


Fig. 7: Definición de la clase tablero.

Clase Celda

La clase Celda fue diseñada con la idea de contener la representación lógica de cada celda que conforma el tablero. En esta clase, los métodos «direccion» y «direccion:» permiten obtener y modificar la dirección a la que apunta una flecha, mientras que «esEspecial» y «esEspecial:» permiten realizar lo propio con la variable de instancia esEspecial, la cual permite especificar si una celda tendrá un potenciador de vida o no mediante un valor lógico o booleano. «vidasExtra» y «vidasExtra:» permiten obtener y modificar la cantidad de vidas extra que una celda otorgará, «rotar90» permite rotar una celda noventa grados en sentido horario luego de que un jugador la abandone, y «desactivarPowerUp» permite convertir una celda en una celda normal, o no potenciadora, luego de que un jugador pase sobre ella, ver Fig. 8.



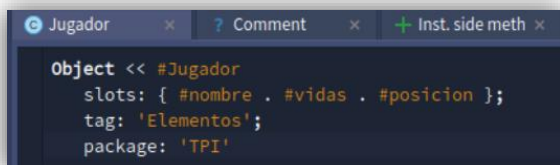
```
Object << #Celda
  slots: { #direccion . #esEspecial . #vidasExtra };
  tag: 'Elementos';
  package: 'TPI'
```

Fig. 8: Definición de clase celda.

Clase Jugador

La clase Jugador fue diseñada con la idea de contener la representación lógica de un jugador y sus atributos, separada de su representación gráfica en el tablero. Atributos que almacenan información importante, como «nombre» y «posicion» pueden ser modificados con métodos homónimos para obtener y modificar sus valores. «vidas» representa la cantidad de vidas que tiene un jugador y puede ser modificado con los métodos «ganarVidas:» y «perderVida».

Cada jugador recibe por defecto una cantidad razonable de vidas para jugar, específicamente tres, a través del método «initialize», y se puede verificar si un jugador está muerto o no a través de «estaMuerto», ver Fig. 9.



```
Object << #Jugador
  slots: { #nombre . #vidas . #posicion };
  tag: 'Elementos';
  package: 'TPI'
```

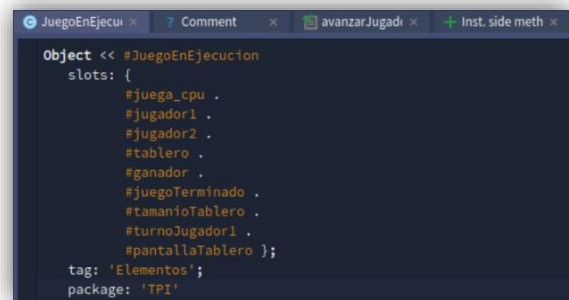
Fig. 9: Definición de clase jugador.

Clase JuegoEnEjecucion

La clase JuegoEnEjecucion fue diseñada como el componente más importante en el diseño del juego, debido a que es la encargada de almacenar muchos componentes lógicos del juego durante toda su ejecución, debido a que necesitamos disponer de ellos durante todo momento para consultarlos y/o modificarlos durante distintas etapas.

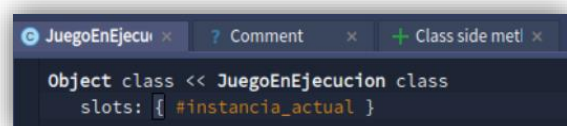
Para poder lograr este objetivo aplicamos dos principios de diseño: el primero, conocido como «Singleton», nos permite restringir el instanciado de esta clase a una sola instancia, mientras que el segundo principio es el almacenamiento de la instancia creada en una variable de clase, de manera que, combinando estas dos prácticas, logramos obtener una instancia que perdura durante toda la ejecución del juego y no es eliminada en un momento poco oportuno por el recolector de basura de Pharo.

La clase tiene muchas variables de instancia, como «jugador1» y «jugador2», que se utilizan para almacenar los objetos correspondientes a los jugadores que participan del juego, con métodos homónimos para obtener o modificar sus valores. El atributo «ganador» tiene un método homónimo para obtener al jugador que ganó la partida, y también se cuenta con otros métodos como «avanzarJugador:», los cuales permiten desplazar por el tablero a un jugador dado, ver Fig. 10 y 11.



```
Object << #JuegoEnEjecucion
  slots: {
    #juega_cpu .
    #jugador1 .
    #jugador2 .
    #tablero .
    #ganador .
    #juegoTerminado .
    #tamanoTablero .
    #turnoJugador1 .
    #pantallaTablero };
  tag: 'Elementos';
  package: 'TPI'
```

Fig. 10: Definición de instancia de la clase JuegoEnEjecucion.



```
Object class << JuegoEnEjecucion class
  slots: { #instancia_actual }
```

Fig. 11: Definición de clase de la clase JuegoEnEjecucion.

2.5 Interfaz Gráfica del Juego

El flujo de construcción de la interfaz gráfica del programa Sistema Dinámico Repulsor fue realizado de forma progresiva. El proceso se inicia con la presentación del menú principal, el cual ofrece al usuario tres opciones fundamentales, ver Fig. 12 y 13. Estas opciones son: Jugar para comenzar la partida, Ver una explicación sobre el juego para conocer las reglas y el funcionamiento, y Salir del juego para finalizar la aplicación.

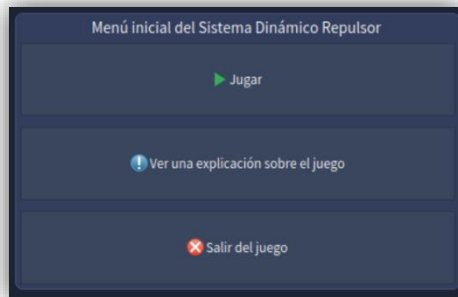


Fig. 12: Ventana menú principal.

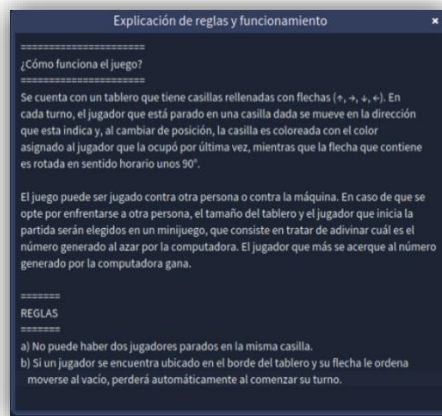


Fig. 13: Ventana explicaciones.

Tras seleccionar la opción Jugar, la interfaz avanza inmediatamente a la etapa de configuración inicial. En este punto, se requiere el ingreso de los nombres de los dos participantes para identificarlos durante el juego, ver Fig. 14 y 15.

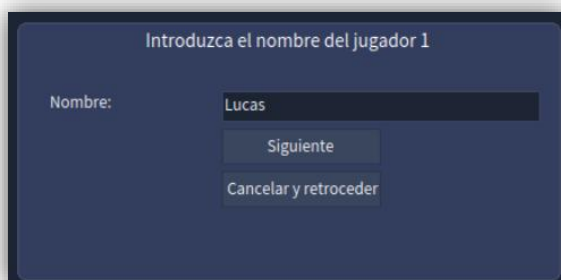


Fig. 14: Ventana selección nombre jugador 1.

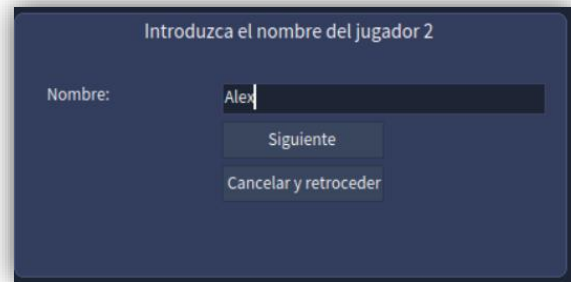


Fig. 15: Ventana sección nombre jugador 2.

La configuración continúa con la ejecución de un primer minijuego esencial para el desarrollo de la partida. La meta de este primer minijuego es determinar las dimensiones del tablero que se utilizará. Ambos participantes introducen un número, y el jugador cuyo número se encuentre más cercano a un valor aleatorio predeterminado gana el derecho a elegir, ver Fig. 16 y 17.

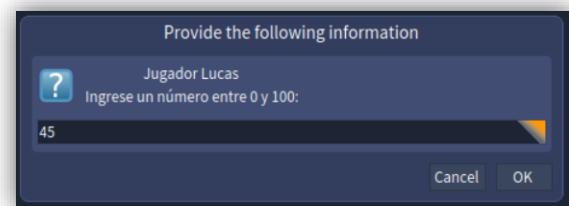


Fig. 16: Mini-juego Tablero jugador 1.

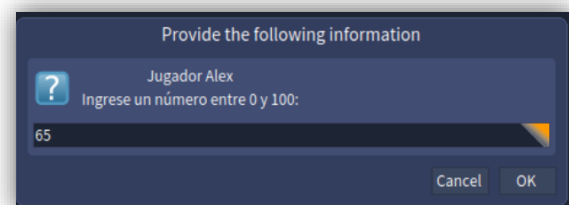


Fig. 17: Mini-juego Tablero jugador 2.

Posteriormente, el jugador ganador selecciona el tamaño del tablero entre las dimensiones de 8x8 10x10, ver Fig. 18.

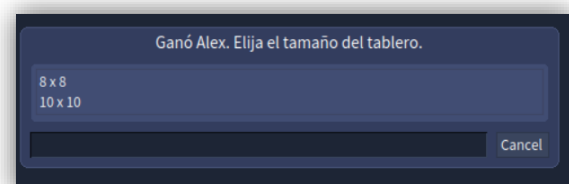


Fig. 18: Ventana selección tablero.

Acto seguido, se procede a la ejecución de un segundo minijuego, cuyo único objetivo es establecer el orden de selección de las posiciones de inicio de los jugadores, ver Fig. 16 y 17. Una vez determinado el orden, la interfaz transiciona a la pantalla de establecimiento de posiciones iniciales. En esta fase, el ganador del segundo minijuego elige su fila y columna de partida. Este proceso es seguido por la selección de la fila y columna de inicio del segundo participante en el tablero, ver Fig. 19 y 20.

Fig. 19: Ganador del 2do Mini-juego, poder de seleccionar primero una posición en el tablero.

Fig. 20: Perdedor del 2do Mini-juego, selecciona segundo una posición en el tablero.

Una vez que el último jugador ha cargado sus coordenadas de inicio, se despliega el tablero principal de juego. Este componente central utiliza flechas orientadas, colores dinámicos, marcadores de posición y casillas para su correcta representación visual, ver Fig. 21. Los jugadores tienen la opción de avanzar a lo largo de este tablero, casilla por casilla por turno, hasta alcanzar la conclusión del juego. Además, al caer en una casilla marcada con el color verde, un jugador obtenga una vida adicional. Inicialmente, a cada jugador se le asignan tres vidas al comienzo de la partida.

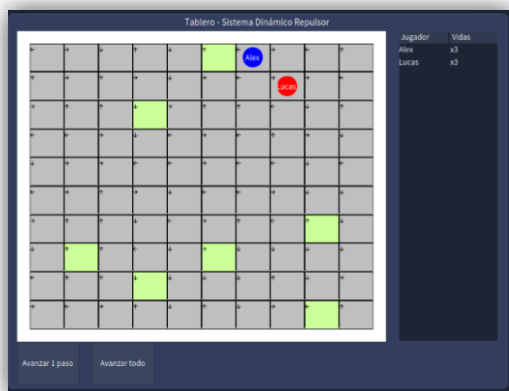


Fig. 21: Ventana del tablero.

El desenlace de la partida ocurre cuando un jugador agota sus vidas, las cuales son descontadas cuando un movimiento saca al jugador de los límites físicos del tablero. La partida finaliza y es ganada por el último jugador que se mantenga en el tablero. Esto sucede después de que el otro participante haya salido del tablero y, consecuentemente, haya perdido todas sus vidas, ver Fig. 22.



Fig. 22: Ventana Fin de Juego.

2.7 Integración y Pruebas del Sistema

En la etapa final se realizaron pruebas exhaustivas del sistema completo. Estas pruebas incluyeron la simulación de recorridos largos, la comprobación del comportamiento del tablero ante múltiples rotaciones, la verificación de expulsiones en los bordes, el funcionamiento de la vida extra, la correcta transición entre pantallas y la estabilidad general durante el desarrollo de la partida. Las verificaciones permitieron confirmar la coherencia del sistema y aseguraron que los elementos implementados respondieran adecuadamente a los objetivos del proyecto.

3. Conclusiones

La redacción del artículo permitió al grupo tomar distancia del proyecto y entender mejor todo el proceso de desarrollo. Al ordenar ideas y justificar decisiones, se evidenció cuánto evolucionó el sistema desde su versión inicial. El análisis ayudó a revisar cada componente, verificar su coherencia y confirmar que los principios de la orientación a objetos estaban bien aplicados.

La escritura también mostró cómo la teoría (abstracción, encapsulamiento, responsabilidades, estado mutable) se integró efectivamente en la práctica al diseñar un sistema complejo. Además,

permitió comprender cómo se añadieron nuevas mecánicas y elementos aleatorios sin perder estabilidad ni coherencia.

El repaso del trabajo reveló la importancia de la iteración, la comunicación entre los integrantes y la revisión constante. Muchos avances surgieron más del proceso de corregir y replantear que del conocimiento técnico inicial.

En conclusión, la ampliación del Sistema Dinámico Repulsor fortaleció la jugabilidad y demostró la capacidad del paradigma orientado a objetos para soportar sistemas dinámicos. Finalmente, el artículo no solo documentó el proyecto, sino que profundizó la comprensión del proceso y destacó el valor de combinar teoría, práctica y colaboración para lograr un sistema sólido y bien diseñado.

Bibliografía

Joyanes Aguilar, L. (1996). Programación orientada a objetos. Universidad Pontificia de Salamanca, Facultad de Informática

Joyanes Aguilar, L. (2008). Fundamentos de programación. Algoritmos, estructura de datos y objetos. Universidad Pontificia de Salamanca, Facultad de Informática.